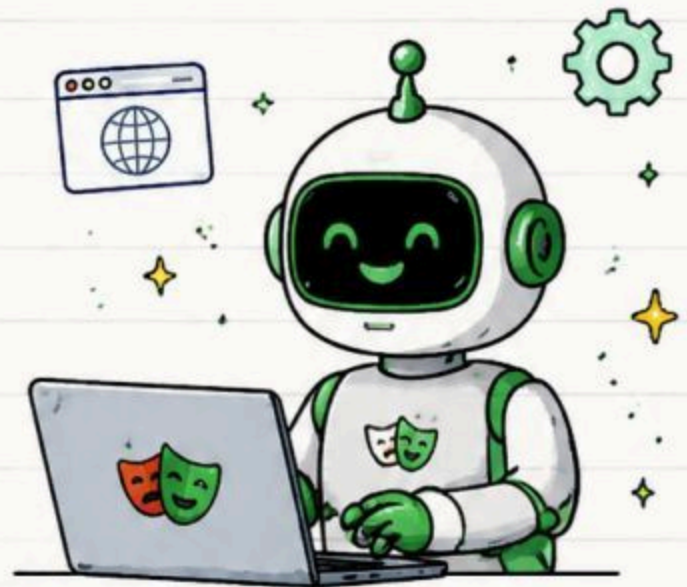


1 What is Playwright?

⇒ "Quality is not an act, it is a habit." ⇒

Playwright is an open-source end-to-end test automation framework developed by Microsoft. It allows us to automate modern web applications across multiple browsers with a single API.



Key Points

- ★ Open-source test automation framework
- ★ Built by Microsoft
- ★ Supports all modern browsers
- ★ Fast, reliable and developer-friendly
- ★ Powerful API with modern features
- ★ Supports UI, API, Mobile testing

Supported Languages

- TypeScript (Recommended)
- JavaScript
- Python
- .NET (C#)
- Java



Playwright

by  Microsoft



Remember:

Playwright helps you write reliable tests with less code and more speed.





2 Why Playwright?

⇒ "Quality is not an act, it is a habit." ⇒

1 Modern & Fast

Built on modern architecture to deliver blazing fast execution.



2 Auto-waiting & Smart Assertions

Automatically waits for elements and ensures stable tests.



3 Parallel Execution

Run tests in parallel across multiple browsers and workers.



4 Cross-browser Support

Test on Chromium, Firefox and WebKit with the same code.



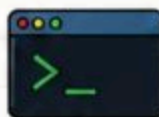
5 Built-in API Testing

Powerful API testing using request context and response validations.



6 Built-in Test Runner

No separate test framework required. Comes with a rich test runner.



7 Developer Friendly

Great DX (Developer Experience), easy debugging & trace viewer.



8 Open-source & Backed by Microsoft

Actively maintained, reliable and trusted by the community.



In Short

- ✓ Fast
- ✓ Reliable
- ✓ Stable
- ✓ Feature-rich
- ✓ Easy to maintain
- ✓ Built for modern web applications

Playwright helps you write better tests with less effort!



Playwright = Productivity + Stability + Reliability 100

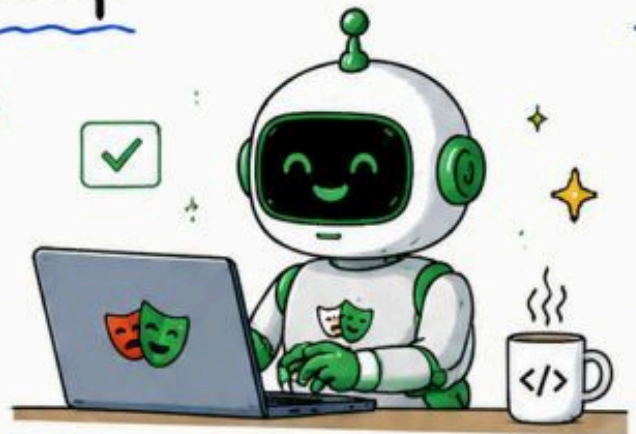


3 Installation & Setup

🌟 "Well begun is half done." 🌟

1 Prerequisites

- Node.js (v16 or higher)
- npm (comes with Node.js)
- VS Code (Recommended)



2 Install Playwright

Open terminal and run:

```
npm init playwright@latest
```



What this command does?

- ✓ Creates a new Playwright project
- ✓ Installs Playwright
- ✓ Downloads browser binaries
- ✓ Sets up configuration files

3 Select Options (During Setup)

- Choose language: **TypeScript**
- Add example tests: **Yes**
- Install browsers: **Yes**
- CI GitHub Actions: **(Optional)**



Tip:

Choose **TypeScript** for better type safety and IDE support.



4 Verify Installation

Run the following command:

```
npx playwright test
```

If everything is set correctly, the sample test should pass! 🟢



5 Project Structure (After Installation)

my-playwright-project/	
tests/	# Test spec files
tests-examples/	# Example tests
pages/	# Page Object Models
playwright-report/	# Test reports
test-results/	# Screenshots, videos, traces
playwright.config.ts	# Playwright config
package.json	# Project dependencies
tsconfig.json	# TypeScript config
README.md	# Project info



Recommended VS Code Extensions



TypeScript
Language
Basics



Playwright
Test for
VSCode



Prettier -
Code
formatter



Indent-Rainbow
Better
Indentation



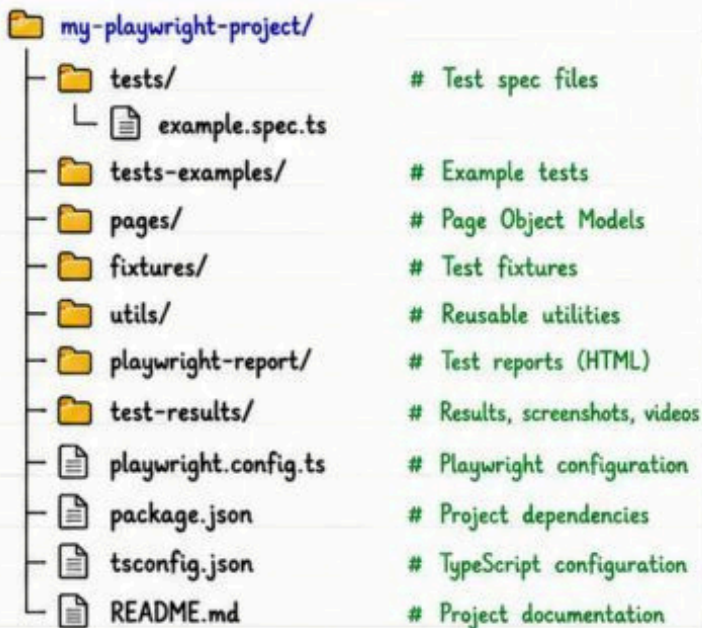
You're Ready!

Your Playwright setup is complete. Better. Let's start writing awesome tests!



⇒ A well-structured project is easy to maintain, scale and collaborate. ⇒

1) Project Structure (Default)

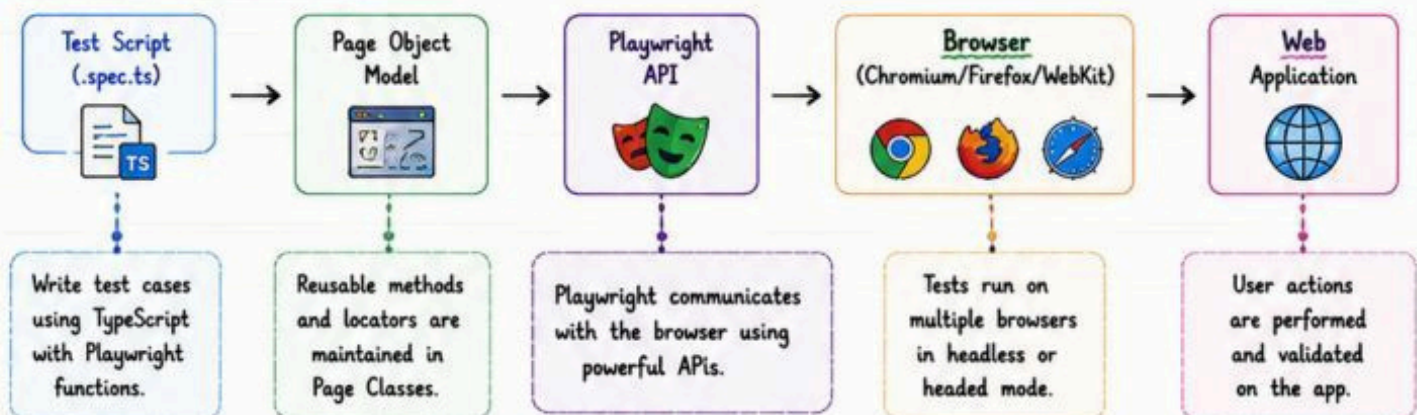


2) Key Folders Explained

- `tests/` → Contains test specification files.
- `pages/` → Page Object Model classes for application pages.
- `fixtures/` → Custom fixtures for setup, teardown and test data.
- `utils/` → Reusable helper functions, constants, and utilities.
- `playwright-report/` → Auto-generated HTML reports.
- `test-results/` → Screenshots, videos, traces, logs and

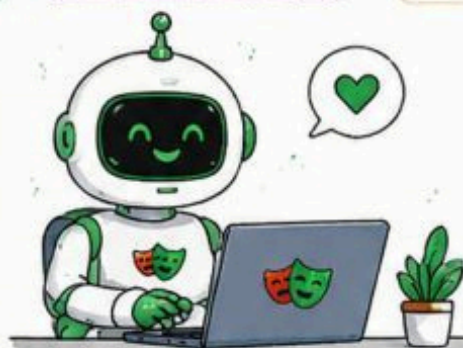


3) Architecture / Flow



4) Benefits of Good Structure

- ✓ Easy maintenance and scalability
- ✓ Reusable code and reduced duplication
- ✓ Better collaboration in teams
- ✓ Easy debugging and faster execution
- ✓ Clear separation of concerns



Pro Tip

Keep your project clean, organized and follow best practices. It saves 10x time in the long run! 100

"Write once, run anywhere."

Playwright supports all modern browsers out of the box.



1) Supported Browsers



Chromium

- Built on Blink engine
- Fast and stable
- Used in Chrome, Edge, Brave, Opera

Recommended



Firefox

- Built on Gecko engine
- Great compatibility
- Good for accessibility testing



WebKit

- Built on WebKit engine
- Powers Safari
- Essential for macOS and iOS testing



Safari



Microsoft Edge

- Built on Chromium
- Same features as Chrome
- Enterprise ready



2) Browser Comparison

Feature	Chromium	Firefox	WebKit (Safari)	Edge (Chromium)
Engine	Blink	Gecko	WebKit	Blink
Speed	Excellent	Very Good	Good	Excellent
Stability	✓	✓	✓	✓
DevTools Support	Excellent	Very Good	Good	Excellent
Mobile Emulation	Excellent	Good	Excellent (iOS)	Excellent
Parallel Execution	✓	✓	✓	✓
Headless Mode	✓	✓	✓	✓
Best For	Web Apps	Compatibility	Apple Ecosystem	Enterprise Apps

3) Run Tests on Multiple Browsers

```
import { test, expect } from '@playwright/test';
test.describe.configure({ mode: 'parallel' });
const browsers = ['chromium', 'firefox', 'webkit'];
for (const browserName of browsers) {
  test(`${browserName} example`, async ({ browser }) => {
    const context = await browser.newContext();
    const page = await context.newPage();
    await page.goto('https://example.com');
    await expect(page).toHaveTitle(/Example/);
    await context.close();
  });
}
```

- List of browsers
- Create new context
- Open new page
- Navigate to URL
- Assertion
- Close context



4) Launch Options

```
const browser = await
chromium.launch({
  headless: false, // true for headless
  slowMo: 50,      // slow down actions
  args: ['--start-maximized'];
});
```

★ Pro Tip

- Always test on at least Chromium, Firefox and WebKit (Safari).
- Real devices + browsers = Better Confidence!



Playwright gives you the power to test everywhere with the same API.

Less code. More coverage. More quality. 100

6 Locators in Playwright

👉 "A good locator makes your tests stable and easy to maintain."

Playwright provides powerful locator engines to identify elements reliably.

Use the most readable and resilient locator. ✓



1) Locator Types

- `getByRole()` - By ARIA role
- `getByText()` - By visible text
- `getByLabel()` - By label text
- `getByPlaceholder()` - By placeholder
- `getByTestId()` - By test id (data-testid)
- `locator()` - CSS / XPath / custom
- `getByAltText()` - By alt text
- `getByTitle()` - By title attribute
- `getByDisplayValue()` - By input value

2) Examples

	<code>getByRole()</code>	<code>await page.getByRole('button', { name: 'Submit' });</code>	// Most preferred
	<code>getByText()</code>	<code>await page.getByText('Sign In');</code>	// By visible text
	<code>getByLabel()</code>	<code>await page.getByLabel('Email');</code>	// By label text
	<code>getByPlaceholder()</code>	<code>await page.getByPlaceholder('Enter email');</code>	// By placeholder
	<code>getByTestId()</code>	<code>await page.locator('css=#main .item');</code>	// CSS SS selector
	<code>locator()</code> - XPath	<code>await page.locator('//div[@class="item"]');</code>	// XPath selector
	<code>getByAltText()</code>	<code>await page.getByAltText('Playwright Logo');</code>	// By alt text
	<code>getByTitle()</code>	<code>await page.getByTitle('Profile');</code>	// By title attribute
	<code>getByDisplayValue()</code>	<code>await page.getByDisplayValue('India');</code>	// By input value

3) Best Practices

- ✓ Prefer `getByRole()`
- ✓ Use `getByLabel()` for forms
- ✓ Use `data-testid` for dynamic UI
- ✓ Avoid brittle CSS & XPath
- ✓ Make locators short and readable
- ✓ Keep locators resilient to UI changes



Good locator = Stable test
Happy tester! ❤️

4) Example Usage

```
import { test, expect } from '@playwright/test';

test('Login test using locators', async ({ page }) => {
  await page.goto('https://example.com/login');
  await page.getByLabel('Email').fill('test@example.com');
  await page.getByLabel('Password').fill('password123');
  await page.getByRole('button', { name: 'Sign In' }).click();
  await expect(page.getByText('Dashboard')).toBeVisible();
});
```

Tips

- ★ `getByRole()` + `name` = Powerful combination!
- ★ Use meaningful labels and test ids.
- ★ Stable tests save hours of debugging!



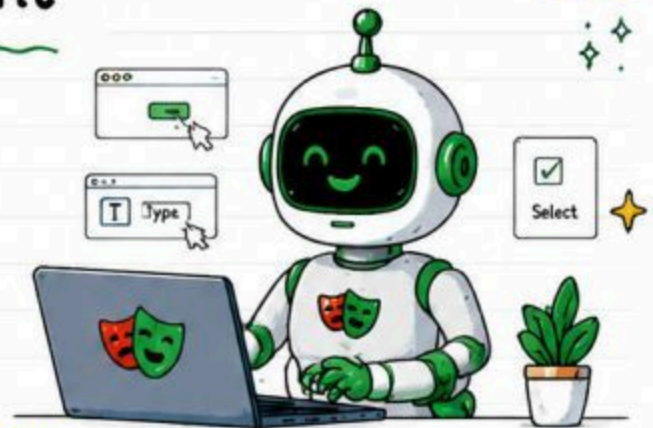
Choose the right locator today, and your tests will thank you tomorrow! **100**

7

Actions in Playwright

"Actions are the way we interact with the application."

Playwright provides rich set of actions to simulate real user interactions.



1 Common Actions

- `click()` - Click on an element
- `dblclick()` - Double click
- `fill()` - Type into input / textarea
- `press()` - Press a key (e.g., Enter, Tab)
- `check()` - Check checkbox
- `unchecked()` - Uncheck checkbox
- `selectOption()` - Select option in dropdown
- `hover()` - Hover over element
- `scrollIntoViewIfNeeded()` - Scroll to element
- `focus()` - Focus on element
- `clear()` - Clear input field
- `screenshot()` - Capture screenshot



2 Action Examples

Action	Example (TypeScript)
<code>click()</code>	<code>await page.locator('#login').click();</code>
<code>fill()</code>	<code>await page.locator('#email').fill('test@example.com');</code>
<code>press()</code>	<code>await page.locator('#search').press('Enter');</code>
<code>check()</code>	<code>await page.locator('#agree').check();</code>
<code>selectOption()</code>	<code>await page.locator('#country').selectOption('India');</code>
<code>hover()</code>	<code>await page.locator('.menu').hover();</code>
<code>dblclick()</code>	<code>await page.locator('#row-1').dblclick();</code>
<code>clear()</code>	<code>await page.locator('#name').clear();</code>

3 Keyboard Shortcuts with `press()`

Enter	Submit / confirm
Tab	Move to next element
Shift + Tab	Move to previous element
Esc	Close dialog / cancel
Ctrl / ⌘ + A	Select all text
Ctrl / ⌘ + C	Copy
Ctrl / ⌘ + V	Paste



4 Real Time Example

```
test('Login flow using actions', async ({ page }) => {
  await page.goto('https://example.com/login');
  await page.locator('#email').fill('test@example.com');
  await page.locator('#password').fill('Password123');
  await page.locator('#remember').check();
  await page.locator('button[type="submit"]').click();
  await expect(page.locator('.dashboard')).toBeVisible();
});
```

Tips

- ★ Prefer user-like actions instead of forcing values.
- ★ Use meaningful waits and assertions after actions.
- ★ Combine actions with locators and assertions for reliable tests.



Great tests don't just check, they **interact** like real users!

100

Assertions & Auto Waiting in Playwright

"Playwright waits for the right time, so you don't have to."

Playwright comes with built-in auto-waiting and powerful assertions to make your tests reliable.



1) Auto Waiting (Built-in)

★ Playwright automatically waits for elements to be:

- ★ ✓ Visible
- ✓ Stable
- ✓ Enabled
- ✓ Receives Events



No need for `Thread.sleep()` or explicit waits in most cases!

2) Assertions Overview

Playwright Test uses `expect()` from `@playwright/test`.

```
import { test, expect } from '@playwright/test';
```

Expectations are auto-waiting by default.

It keeps retrying until the condition is met or timeout occurs.

3) Common Assertions

Assertion	What it does	Example (TypeScript)
✓ <code>toBeVisible()</code>	Element is visible	<code>await expect(locator).toBeVisible();</code>
✓ <code>toBeHidden()</code>	Element is hidden	<code>await expect(locator).toBeHidden();</code>
✓ <code>toBeEnabled()</code>	Element is enabled	<code>await expect(locator).toBeEnabled();</code>
✓ <code>toBeDisabled()</code>	Element is disabled	<code>await expect(locator).toBeDisabled();</code>
✓ <code>toHaveText()</code>	Text matches	<code>await expect(locator).toHaveText('Dashboard');</code>
✓ <code>toContainText()</code>	Text contains value	<code>await expect(locator).toContainText('Welcome');</code>
✓ <code>toHaveValue()</code>	Input has value	<code>await expect(locator).toHaveValue('admin');</code>
✓ <code>toHaveAttribute()</code>	Element has attribute	<code>await expect(locator).toHaveAttribute('type', 'submit');</code>
✓ <code>toHaveLength()</code>	Number of elements	<code>await expect(locator).toHaveLength(5);</code>

4) Soft Assertions

Soft assertions will not stop test on failure.

```
await expect.soft(locator).toBeVisible();
await expect.soft(locator).toHaveText('Hello');
```

Use when you want to collect multiple failures in a single run.



5) Custom Message

You can add custom error messages.

```
await expect(locator).toHaveText('Success', {
  message: 'Expected success message not found!'
});
```

6) Example

```
import { test, expect } from '@playwright/test';

test('Login validation', async ({ page }) => {
  await page.goto('https://example.com/login');
  await page.locator('#username').fill('admin');
  await page.locator('#password').fill('password123');
  await page.locator('button[type="submit"]').click();

  await expect(page.locator('.dashboard')).toBeVisible();
  await expect(page.locator('.welcome')).toContainText('Welcome, admin');
  await expect(page.locator('.logout')).toBeEnabled();
});
```

What happens?

- ✓ Playwright waits for element automatically.
- ✓ Assertions retry until condition is true or timeout (5s by default).
- ✓ If condition not met → Test fails.



Pro Tip

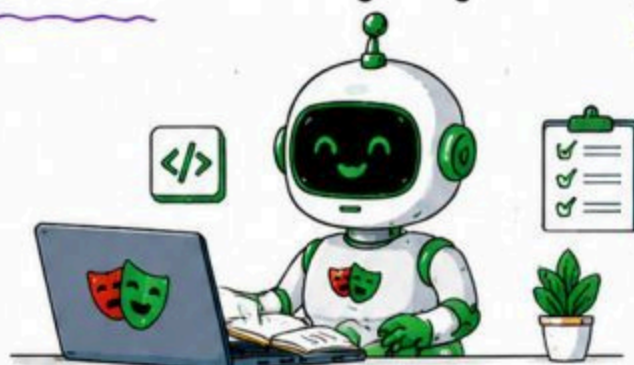
- Rely on Playwright's auto waiting and assertions. Avoid `Thread.sleep()` unless absolutely necessary.



Good assertions + Auto waiting = Stable, Reliable & Fast Tests! **100**

"Good design makes tests easy to write, maintain and scale."

POM helps in maintaining a clean separation between test logic and page implementation. Fixtures help in setup and teardown.



1) POM - What & Why?

- ★ Each Page is represented as a Class.
- ★ Locators and actions are stored in Page Classes.
- ★ Test files call methods from Page Classes.
- ★ Reduces code duplication and improves maintainability.
- ★ Easy to handle UI changes in one place.



3) Example - Page Class (login.page.ts)

```
import { Page, expect } from '@playwright/test';

export class LoginPage {
  readonly page: Page;

  constructor(page: Page) {
    this.page = page;
  }

  async goto() {
    await this.page.goto('https://example.com/login');
  }

  async login(username: string, password: string) {
    await this.page.locator('#username').fill(username);
    await this.page.locator('#password').fill(password);
    await this.page.locator('button[type="submit"]').click();
  }

  async getErrorMessage() {
    return await this.page.locator('.error').textContent();
  }
}
```

Benefits

- ✓ Reusable
- ✓ Maintainable
- ✓ Readable

2) POM Structure Example

```
my-playwright-project/
├── pages/
│   ├── login.page.ts      # Page Object Classes
│   └── dashboard.page.ts
├── tests/
│   ├── login.spec.ts      # Test files
│   └── base.fixture.ts    # Custom Fixtures
├── fixtures/
│   ├── base.fixture.ts    # Custom Fixtures
│   └── playwright.config.ts # Playwright configuration
```

4) Example - Using POM in Test (login.spec.ts)

```
import { test, expect } from '@playwright/test';
import { LoginPage } from '../pages/login.page';

test('Login with valid credentials', async ({ page }) => {
  const loginPage = new LoginPage(page);
  await loginPage.goto();
  await loginPage.login('admin', 'admin123');
  await expect(page.locator('.dashboard')).toBeVisible();
});

test('Login with invalid credentials', async ({ page }) => {
  const loginPage = new LoginPage(page);
  await loginPage.goto();
  await loginPage.login('admin', 'wrongpass');
  await expect(await loginPage.getErrorMessage())
    .toContain('Invalid credentials');
});
```



5) Fixtures - What & Why?

- ★ Fixtures provide a way to set up preconditions and clean up after tests.
- ★ Avoids repetitive setup/teardown in every test.
- ★ Supports sharing data and objects across tests.



6) Example - Custom Fixture (base.fixture.ts)

```
import { test as base, BrowserContext, Page } from '@playwright/test';
import { LoginPage } from '../pages/login.page';

type MyFixtures = {
  LoginPage: LoginPage;
};

export const test = base.extend<MyFixtures>({
  LoginPage: async ({ page }, use) => {
    const loginPage = new LoginPage(page);
    await use(loginPage); // provide to tests
  },
});

export { expect } from '@playwright/test';
```

7) Using Fixture in Tests

```
import { test, expect } from '../fixtures/base.fixture';

test('Login using fixture', async ({ LoginPage }) => {
  await LoginPage.goto();
  await LoginPage.login('admin', 'admin123');
  await expect(LoginPage.page.locator('.dashboard'))
    .toBeVisible();
});
```



Pro Tip

- POM + Fixtures = Clean, Scalable and Maintainable Tests!
- ✓ Easy to update
 - ✓ Easy to reuse
 - ✓ Easy to scale



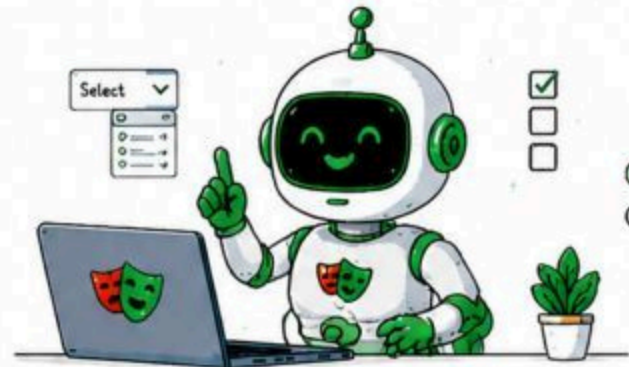
Write once, reuse everywhere!

Build smart tests with POM & Fixtures. **100**

"Handle elements smartly,
Test reliably, every time."

Web apps are full of user inputs.

Playwright makes handling them simple and powerful.



1) Dropdowns (Select Boxes)

Use `selectOption()` to select by value, label or index.

Method	Description	Example
<code>selectOption({ value })</code>	Select by value attribute	<code>await page.locator("#country").selectOption({ value: 'IN' });</code>
<code>selectOption({ label })</code>	Select by visible text	<code>await page.locator("#country").selectOption({ label: 'India' });</code>
<code>selectOption({ index })</code>	Select by index (0 based)	<code>await page.locator("#country").selectOption({ index: 2 });</code>

Dropdown Example

```
test('Select country from dropdown', async ({ page }) => {
  await page.goto('https://example.com/register');
  await page.locator("#country").selectOption({ label: 'India' });
  await expect(page.locator("#country")).toHaveValue('IN');
});
```



3) Radio Buttons

Use `check()` to select a radio button.

Only one radio button in a group can be selected.

Select by value

```
await page.locator('input[name="gender"]')
  .check('female');
```

Select by label text

```
await page.getByLabel('Female').check();
```

Radio Button Example

```
test('Radio button example', async ({ page }) => {
  await page.goto('https://example.com/profile');
  await page.getByLabel('Female').check();
  await expect(
    page.locator('input[name="gender"][value="female"]')
  ).toBeChecked();
});
```

☐ Male ☒ Female ☐ Other

2) Checkboxes

Use `check()`, `unchecked()` and `isChecked()` methods.

Method	Description
<code>check()</code>	Checks the checkbox (selects it)
<code>unchecked()</code>	Unchecks the checkbox
<code>isChecked()</code>	Returns true if checkbox is checked

Checkbox Example

```
test('Checkbox example', async ({ page }) => {
  await page.goto('https://example.com/preferences');
  const newsletter = page.locator("#newsletter");
  await newsletter.check();
  expect(await newsletter.isChecked()).toBeTruthy();
  await newsletter.uncheck();
  expect(await newsletter.isChecked()).toBeFalsy();
});
```



4) Handling Multiple Elements

When multiple checkboxes/radios exist, use `.nth()`, `.all()` or `.filter()`.

Get all checkboxes

```
const checkboxes = page.locator('input[type="checkbox"]');
const count = await checkboxes.count();
for (let i = 0; i < count; i++) {
  await checkboxes.nth(i).check();
}
```

Filter example

```
await page.locator('input[type="checkbox"]').filter({ hasText: 'I agree' }).check();
```



5) Verify Selected Options

- ★ `toHaveValue()` - for dropdowns
- ★ `isChecked()` - for checkboxes/radios
- ★ `toBeChecked()` - Playwright assertion



Pro Tips

- ✓ Prefer `getByRole()` with name for better reliability.
- ✓ Avoid using index unless necessary.
- ✓ Use meaningful locators (label, placeholder, role).
- ✓ Always assert after performing actions.



Common Pitfalls

- ✗ Selecting wrong element due to weak locator.
- ✗ Not waiting for element to be visible.
- ✗ Assuming default selection without asserting.
- ✗ Using index where order may change.

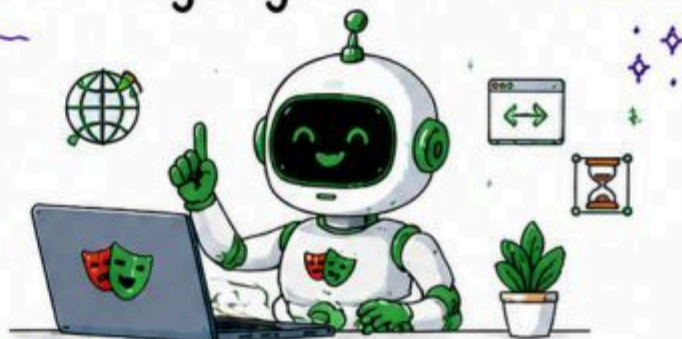


Handle every input with confidence - Clean locators, smart actions, reliable tests! **100**

"Control the flow, handle the context, make your tests rock solid."

Web pages don't stay still!

Playwright provides powerful APIs to navigate, switch contexts and wait for the right conditions.



1) Navigation Methods

Use page object methods to navigate.

Method	Description
<code>page.goto(url)</code>	Navigate to a URL
<code>page.goBack()</code>	Go back in browser history
<code>page.goForward()</code>	Go forward in browser history
<code>page.reload([options])</code>	Reload the page

Example

```
await page.goto('https://example.com');
await page.goto('https://example.com/login');
await page.goBack();
await page.goForward();
await page.reload();
```



2) Frames (iframes)

Frames allow you to interact with content inside an iframe.

Method	Description
<code>frameLocator(selector)</code>	Locate iframe
<code>locator(selector).contentFrame()</code>	Get frame from iframe element
<code>frame.locator(selector)</code>	Locate element inside frame

Example

```
// Using frameLocator
const frame = page.frameLocator('iframe[src*=example]');
await frame.locator('#username').fill('admin');

// Using contentFrame()
const frame = await page.locator('iframe').contentFrame();
await frame.locator('#password').fill('password123');
```



3) Waits Overview

Playwright auto waits by default. Still, explicit waits are useful in some cases.

Wait Type	When to Use
Auto Wait	Default for actions & assertions
<code>waitForSelector()</code>	Wait for element to appear/disappear
<code>waitForLoadState()</code>	Wait for page load state (load, domcontentloaded, networkidle)
<code>waitForTimeout()</code>	Fixed timeout (use minimal, not preferred)
<code>expect()</code>	Auto retry until condition is met or timeout

Best Practices

- ✓ Rely on auto waits & `expect()` as much as possible.
- ✓ Use `waitForSelector()` only when absolutely required.
- ✓ Avoid `waitForTimeout()` unless for debugging.



Pro Tips

- ★ Prefer `expect()` over explicit waits.
- ★ Use `networkidle` only when API calls are critical.
- ★ Keep timeouts reasonable (default is 30s).



Common Pitfalls

- ✗ Overusing `waitForTimeout()`.
- ✗ Not handling frames correctly.
- ✗ Waiting for wrong load state.
- ✗ Ignoring auto waiting & assertions.



4) Waits Examples

```
// Wait for element to be visible
await page.waitForSelector('#dashboard', { state: 'visible' });

// Wait for element to be hidden
await page.waitForSelector('.loader', { state: 'hidden' });

// Wait for page load state
await page.waitForLoadState('domcontentloaded');
await page.waitForLoadState('networkidle');

// Wait using expect (auto wait)
await expect(page.locator('#title')).toHaveText('Dashboard');

// Hard wait (use carefully)
await page.waitForTimeout(2000); // 2 seconds
```



Quick Reference

Goto	<code>page.goto(url)</code>
Back / Forward	<code>page.goBack()</code> / <code>page.goForward()</code>
Reload	<code>page.reload()</code>
Frame	<code>frameLocator()</code> / <code>contentFrame()</code>
Wait Element	<code>waitForSelector()</code>
Wait Page	<code>waitForLoadState()</code>
Assertion Wait	<code>expect(locator).to...</code>



Control navigation. Handle context. Wait smart. Build reliable & stable tests! **100**

"Great tests are not just written,
they are built with good practices."

Following best practices ensures your tests are
clean, maintainable, reliable and easy to debug.



1 Best Practices

- ★ Follow Page Object Model (POM) to separate test code from page actions.
- ★ Use meaningful names for locators, variables, tests and methods.
- ★ Use data-driven approach for multiple sets of data.
- ★ Avoid hard-coded values. Use config files and environment variables.
- ★ Use explicit waits and assertions. Avoid unnecessary hard waits.
- ★ Keep locators resilient and prefer role/text/test id.
- ★ Keep tests independent and idempotent.
- ★ Generate and review reports after every run.
- ★ Version control your tests (Git).



2 Locator Best Practices

Prefer these locator priority order:

- 1 getByRole() (Most reliable)
- 2 getByTestId()
- 3 getByLabelText()
- 4 getByPlaceholder()
- 5 getByText() / getByTitle()
- 6 CSS / XPath (Last resort)

3 Avoid These

- ✗ Avoid `page.waitForTimeout()` for waiting.
- ✗ Avoid brittle CSS/XPath.
- ✗ Avoid too many assertions in one test.
- ✗ Avoid dependent tests.
- ✗ Avoid global state & shared data.
- ✗ Avoid testing third-party issues.



4 Good Assertion Practices

- ✓ Assert the outcome, not the steps.
- ✓ Use `toHave*` matchers for state verification.
- ✓ Use `expect.poll()` for dynamic data.
- ✓ Use soft assertions when you want to continue after failure.
- ✓ Keep assertions simple and meaningful.



5 Useful Assertion Examples

Assertion	Example (TypeScript)
Visible	<code>await expect(locator).toBeVisible();</code>
Hidden	<code>await expect(locator).toBeHidden();</code>
Text	<code>await expect(locator).toHaveText('Dashboard');</code>
Value	<code>await expect(locator).toHaveValue('admin');</code>
Enabled	<code>await expect(locator).toBeEnabled();</code>
Have Count	<code>await expect(locator).toHaveCount(3);</code>
Contain Text	<code>await expect(locator).toContainText('Welcome');</code>

Remember

Good assertions
make failures
easy to understand.



4 Debugging Techniques

- 💡 Use PW debug mode:
`npx playwright test --debug`
- 💡 Use `page.pause()` to pause execution.
- 💡 Use `page.screenshot()` and `page.video()` to capture issues.
- 💡 Use browser devtools (Playwright Inspector).



7 Debugging Example

```
// Run in debug mode
npx playwright test --debug

// In test
await page.goto('https://example.com');
await page.pause(); // Pause and inspect
await page.screenshot({ path: 'debug.png' });
await page.locator('#login').fill('admin');
await page.locator('#password').fill('admin123');
await page.locator('button[type="submit"]').click();
```



8 Helpful Config Practices

- ✓ Set `baseURL` in `playwright.config.ts`
- ✓ Use projects for multiple browsers.
- ✓ Set retries for flaky tests.
- ✓ Use timeouts wisely.
- ✓ Store test data in JSON/CSV files.
- ✓ Use environment variables for different environments.



9 Playwright CLI Commands

Command	Description
<code>npx playwright test</code>	Run all tests
<code>npx playwright test tests/login.spec.ts</code>	Run specific test file
<code>npx playwright test --headed</code>	Run in headed mode (visible browser)
<code>npx playwright test --debug</code>	Run in debug mode
<code>npx playwright show-report</code>	Open HTML report
<code>playwright codegen</code>	Open Codegen tool

10 Folder Structure Best Practice

tests/		
pages/		
login.page.ts	→	Page Object classes
dashboard.page.ts		
tests/specs/		
login.spec.ts	→	Test spec files
dashboard.spec.ts		
fixtures/		
base.fixture.ts	→	Custom fixtures
data/		
testdata.json	→	Test data files
playwright.config.ts		
package.json	→	Config & dependencies



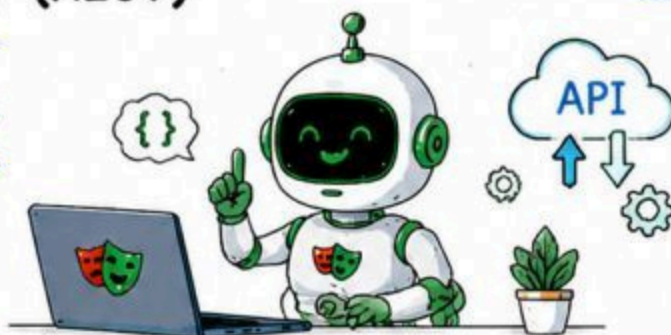
Good practices today, reliable automation tomorrow!



Happy Testing! 100

"APIs power your application.
Test them with speed and confidence."

Playwright provides powerful API testing capabilities using the built-in request context.



1 API Testing - Why?

- ★ Faster than UI tests.
- ★ Validates backend logic.
- ★ Early bug detection.
- ★ Not dependent on UI changes.
- ★ Easier test data setup and maintenance.



2 Request Context

Create a request context to send API requests.

```
import { request } from '@playwright/test';

const apiContext = await request.newContext({
  baseURL: 'https://reqres.in/api',
  extraHTTPHeaders: {
    'Content-Type': 'application/json',
    'Accept': 'application/json'
  }
});
```

3 HTTP Methods

Method	Use
GET	Retrieve data
POST	Create new resource
PUT	Update entire resource
PATCH	Update partial resource
DELETE	Delete resource

4 Common API Assertions

Assertion	Example (TypeScript)
✓ Status Code	<code>expect(response.status()).toBe(200);</code>
✓ Response Body	<code>const body = await response.json(); expect(body.data.id).toBe(1);</code>
✓ Headers	<code>expect(response.headers()['content-type']) .toContain('application/json');</code>
✓ Response Time	<code>expect(response.timing().responseEnd - response.timing().startTime).toBeLessThan(1000);</code>
✓ Array Length	<code>expect(body.data).toHaveLength(6);</code>
✓ JSON Path / Value	<code>expect(body.support.url).toBe('https://reqres.in/#support-heading');</code>
✓ Schema Validation	<code>// Use libraries like ajv for JSON schema validation</code>

5 Example - GET Request

```
test('Get user list', async ({}) => {
  const response = await apiContext.get('/users?page=2');
  expect(response.status()).toBe(200);

  const body = await response.json();
  expect(body.page).toBe(2);
  expect(body.total_pages).toBe(2);
  expect(body.data).toHaveLength(6);
});
```



6 Example - POST Request

```
test('Create a user', async ({}) => {
  const payload = { name: 'morpheus', job: 'leader' };
  const response = await apiContext.post('/users', { data: payload });
  expect(response.status()).toBe(201);

  const body = await response.json();
  expect(body.name).toBe(payload.name);
  expect(body.id).toBeTruthy(); // id is created by server
});
```



7 Example - PUT / PATCH / DELETE

PUT - Update	PATCH - Partial Update	DELETE - Remove
<code>// Update user completely const payload = { name: 'neo', job: 'one' }; const response = await apiContext.put('/users/2', { data: payload }); expect(response.status()).toBe(200);</code>	<code>// Update user partially const payload = { job: 'matrix' }; const response = await apiContext.patch('/users/2', { data: payload }); expect(response.status()).toBe(200);</code>	<code>const response = await apiContext.delete('/users/2'); expect(response.status()) .toBe(204);</code>



8 Useful Tips

- ✓ Reuse request context for performance.
- ✓ Use baseURL to avoid repeating the host.
- ✓ Validate both status code and response body.
- ✓ Use .json(), .text(), .headers() as needed.
- ✓ Keep payloads in separate data files (JSON).



9 Common Pitfalls

- ✗ Hardcoding IDs - use dynamic data when possible.
- ✗ Not validating response body structure.
- ✗ Ignoring error responses (4xx, 5xx).
- ✗ Not handling auth tokens securely.
- ✗ Making dependent calls without cleanup.



10 Pro Tips

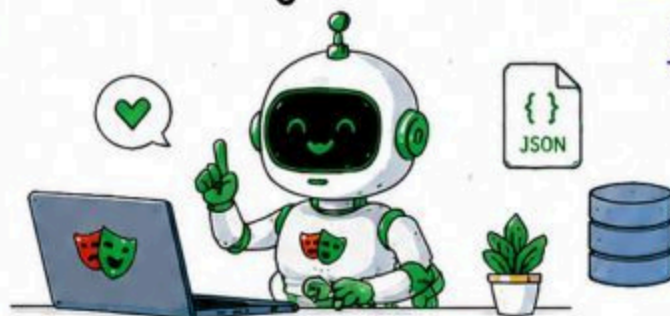
- ★ Use environment variables for baseURL and tokens.
- ★ Log requests/responses in debug mode.
- ★ Use fixtures to share auth tokens across tests.
- ★ Validate response schema using libraries (e.g., ajv).
- ★ Combine API tests with UI tests for end-to-end confidence.



Test APIs **fast**, validate responses **smart**, deliver quality with **confidence!** 100

"Good assertions catch bugs early."
Good data covers every scenario."

Strong assertions + Smart test data =
Reliable and meaningful tests.



1 Playwright Assertions - Overview

Use Playwright's `expect()` API for clear and powerful assertions.

Type	Common Assertions (Examples)
Visibility	<code>toBeVisible()</code> , <code>toBeHidden()</code>
Text	<code>toHaveText()</code> , <code>toContainText()</code>
Value / Attribute	<code>toHaveValue()</code> , <code>toHaveAttribute()</code>
State	<code>toBeEnabled()</code> , <code>toBeDisabled()</code> , <code>toBeChecked()</code>
Count	<code>toHaveCount()</code>
Navigation	<code>toHaveURL()</code> , <code>toHaveTitle()</code>

4 Custom Assertions (Reusable)

Create custom matchers for project specific validations.

```
// example: toBeValidEmail()
import { expect as baseExpect } from '@playwright/test';

export const expect = baseExpect.extend({
  toBeValidEmail: async ({ received }, use) => {
    const pass = /.+@.+\.+/.test(received);
    await use(pass, { message: () =>
      'expected $$received to be a valid email' });
  }
});

// Usage
await expect('test@example.com').toBeValidEmail();
```

2 Assertion Examples

```
import { test, expect } from '@playwright/test';

test('Assertion examples', async ({ page }) => {
  await page.goto('https://example.com');

  // Visibility
  await expect(page.locator('#title')).toBeVisible();

  // Text
  await expect(page.locator('h1')).toHaveText('Dashboard');

  // Value
  await expect(page.locator('input[name="email"]'))
    .toHaveValue('test@example.com');

  // State
  await expect(page.locator('#checkboxes')).toBeChecked();

  // Count
  await expect(page.locator('.item')).toHaveCount(5);
});
```

3 Soft Assertions

Soft assertions continue execution even if an assertion fails.

```
import { test, expect } from '@playwright/test';

test('Soft assertions example',
  async ({ page, expect }) => {
    await page.goto('https://example.com');

    await expect.soft(page.locator('h1'))
      .toHaveText('Dashboard');
    await expect.soft(page.locator('#status'))
      .toBeVisible();

    // Test will continue and report all failures
  });
```

💡 Use soft assertions in smoke tests or when you want all failures in one run.

5 Test Data Management - Approaches

- ★ Hard-coded data - Simple but not scalable.
- ★ External files (JSON/CSV/Excel) - Scalable & maintainable.
- ★ Database - For large dynamic datasets.
- ★ Environment variables - For env-specific data.

6 Using JSON Test Data

Store test data in JSON file.

```
testdata/
├── users.json
└── loginData.json
```

```
{
  "users": {
    { "name": "admin", "password": "admin123" },
    { "name": "user", "password": "user123" }
  }
}
```

```
// Read JSON in test
import users from '../testdata/users.json' assert { type: 'json' };

test('Login with JSON data', async ({ page }) => {
  const user = users.user(0);
  await page.fill('#username', user.name);
  await page.fill('#password', user.password);
});
```

7 Using CSV Test Data

CSV is great for tabular data and data-driven tests.

data.csv

```
username,password,role
admin,admin123,admin
user,user123,standard
```

```
import fs from 'fs';
import { parse } from 'csv-matrix';

const csvData = fs.readFileSync('testdata/data.csv');
const records = parse(csvData, { columns: true });

test('Data driven login with CSV', async ({ page }) => {
  for (const row of records) {
    await page.goto('https://example.com');
    await page.fill('#username', row.username);
    await page.fill('#password', row.password);
    // perform login
  }
});
```

8 Using Environment Variables

Store environment specific data like URLs, tokens, credentials.

```
// .env file
BASE_URL=https://staging.example.com
API_KEY=abcd1234

// In test
import 'dotenv/config';
const baseUrl = process.env.BASE_URL;
await page.goto(baseUrl);
```

8 Best Practices

- ✓ Use strong assertions for validation.
- ✓ Keep test data external, not in code.
- ✓ Use meaningful & reusable test data.
- ✓ Mask sensitive data (passwords, tokens).
- ✓ Keep data files version controlled.



Pro Tip

Good data + Strong assertions
= Confident releases!

Common Pitfalls

- ✗ Overusing hardcoded data.
- ✗ Weak assertions (e.g., text includes).
- ✗ Sharing test data between tests.
- ✗ Not handling dynamic data.



Pro Tip

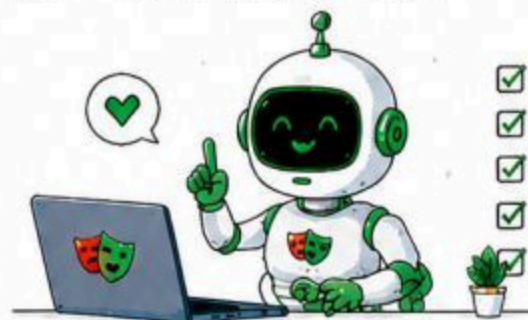
- ★ Use `test.describe.parallel()` with data sets for speed.
- ★ Use factories to create dynamic test data.
- ★ Clean up test data after execution.
- ★ Log useful data only when needed.



Strong assertions. Smart data. Reliable tests. Confident delivery! **100**

"Think like a user, automate like a pro,
deliver quality every step of the way."

An end-to-end test simulates real user journey
from start to finish.

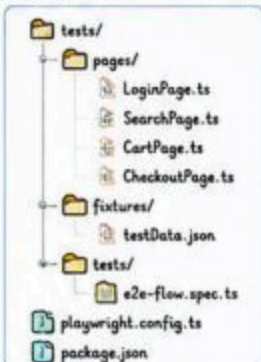


1 Real World Scenario

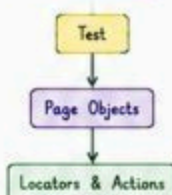
Scenario: Verify that a user can login, search a product, add to cart, checkout and logout successfully.



2 Project Structure (Example)



Using Page Object Model
for maintainability.



3 Test Data (testData.json)

```
{
  "user": {
    "email": "demo.user@example.com",
    "password": "StrongPass@123"
  },
  "product": {
    "searchText": "wireless headphones",
    "expectedProduct": "Wireless Headphones"
  }
}
```



4 End-to-End Test Flow (e2e-flow.spec.ts)

```
import { test, expect } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';
import { SearchPage } from '../pages/SearchPage';
import { CartPage } from '../pages/CartPage';
import { CheckoutPage } from '../pages/CheckoutPage';
import testData from '../fixtures/testData.json';

test('E2E: Login, Search, Add to Cart, Checkout & Logout', async ({ page }) => {
  const loginPage = new LoginPage(page);
  const searchPage = new SearchPage(page);
  const cartPage = new CartPage(page);
  const checkoutPage = new CheckoutPage(page);

  // 1. Login
  await loginPage.goto();
  await loginPage.login(testData.user.email, testData.user.password);
  await expect(page).toHaveURL(/dashboard/);

  // 2. Search Product
  await searchPage.search(testData.product.searchText);
  await expect(searchPage.getProductTitle()).toContainText(testData.product.expectedProduct);

  // 3. Add to Cart
  await searchPage.addProductToCart();
  await expect(page.locator('.cart-count')).toHaveText('1');

  // 4. Checkout
  await cartPage.gotoCart();
  await expect(cartPage.getCartItemNames()).toContainText(testData.product.expectedProduct);
  await cartPage.proceedToCheckout();
  await checkoutPage.fillShippingDetails();
  await checkoutPage.placeOrder();
  await expect(checkoutPage.getOrderConfirmation()).toBeVisible();

  // 5. Logout
  await page.locator('#data-test-logout').click();
  await expect(page).toHaveURL(/login/);
});
```

5 Key Highlights

- ★ Uses Page Object Model for clean and reusable code.
- ★ External test data for flexibility (JSON).
- ★ Validates every important step.
- ★ Assertions at business critical points.
- ★ Easy to maintain and scale.



6 Reporting & Screenshots

- ✓ Playwright auto captures screenshots on failure.
- ✓ HTML reports with detailed logs.
- ✓ Trace viewer for debugging failed tests.
- ✓ CI/CD integration ready.



```
// playwright.config.ts
export default defineConfig({
  use: {
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure'
  },
  reporter: [['html'], ['list']]
});
```

Best Practices

- ✓ Keep tests independent and idempotent.
- ✓ Use stable locators (data-test, role).
- ✓ Keep test data external.
- ✓ Follow AAA pattern (Arrange, Act, Assert).
- ✓ Run tests in headless mode in CI.



Pro Tips

- ★ Use test.step() for better reporting.
- ★ Group related actions in page object methods.
- ★ Use fixtures for login once per test file.
- ★ Keep tests fast and deterministic.



Common Pitfalls

- ✗ Hard coding test data.
- ✗ Using brittle locators (CSS/XPath).
- ✗ Skipping assertions.
- ✗ Creating dependent test cases.

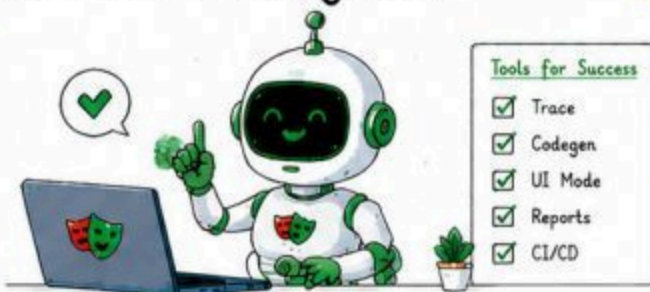


Automate the journey. Deliver confidence. Quality is our destination!

100

"Good tooling + Smart configs + CI/CD
= Stable, scalable & efficient testing."

Leverage Playwright's powerful tooling and integrate seamlessly into your delivery pipeline.



1) Playwright Config File (playwright.config.ts)

```
import { defineConfig, devices } from '@playwright/test';
export default defineConfig({
  testDir: './tests',
  timeout: 30 * 1000,
  expect: { timeout: 5000 },
  fullyParallel: true,
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 1 : undefined,
  reporter: [['html'], ['line']],
  use: {
    baseURL: 'https://example.com',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    headless: true,
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit', use: { ...devices['Desktop Safari'] } },
  ]
});
```



2) Built-in Tooling

Tool	Purpose
Codegen	Generate locators and test scripts by recording actions.
UI Mode	Run tests in interactive mode with live debugging.
Trace Viewer	Visualize every step, network, logs and snapshots.
HTML Report	Rich, interactive test reports.
Test Runner	Fast, reliable and parallel execution.
Inspector	Inspect elements and pick locators.

3) Using Codegen

Generate tests while performing actions in browser.

`npx playwright codegen https://example.com`



- ☆ Opens browser and records your actions.
- ☆ Generates locator-friendly code.
- ☆ Great for quick test authoring.

4) Using UI Mode

`npx playwright test --ui`

- ✓ Interactive UI to run, debug and inspect tests.
- ✓ Step through tests, view logs and DOM.
- ✓ Modify and re-run tests on the fly.



5) Trace Viewer

`npx playwright show-trace trace.zip`

- ✓ View DOM snapshots, screenshots and network.
- ✓ Step-by-step timeline of test execution.
- ✓ Pinpoint failures quickly.



6) HTML Report

`npx playwright show-report`

- ✓ Beautiful, interactive and filterable reports.
- ✓ View test results, retries, errors and attachments.
- ✓ Share reports with team easily.



7) Parallel Execution

- ☆ Run tests in parallel across multiple browsers.
- ☆ Speeds up execution and improves feedback cycle.
- ☆ Controlled via workers and projects in config.



8) Environment Configuration

- ✓ Use .env file to manage environment specific data.

```
BASE_URL=https://staging.example.com
API_KEY=abc123
USERNAME=testuser
PASSWORD=test@123
```

```
// In config or test
import * as dotenv from 'dotenv';
dotenv.config();

const baseUrl = process.env.BASE_URL;
```



9) CI/CD Integration with GitHub Actions

Example: `.github/workflows/playwright.yml`

```
name: Playwright Tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Use Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '18'
      - name: Install dependencies
        run: npm ci
      - name: Install Playwright Browsers
        run: npx playwright install --with-deps
      - name: Run Playwright tests
        run: npx playwright test
      - name: Upload Report
        uses: actions/upload-artifact@v4
        with:
          name: playwright-report
          path: playwright-report/
```



10) Useful CLI Commands

Command	Description
<code>npx playwright test</code>	Run all tests
<code>npx playwright test --project=chromium</code>	Run tests on specific project
<code>npx playwright test --headed</code>	Run tests in headed mode
<code>npx playwright show-report</code>	Open HTML report
<code>npx playwright show-trace trace.zip</code>	Open trace viewer
<code>npx playwright codegen url</code>	Open codegen for given URL
<code>npx playwright test --ui</code>	Open UI mode

Common Pitfalls

- ✗ Hard coding locators.
- ✗ Not waiting for elements.
- ✗ Ignoring test isolation.
- ✗ Overusing retries.
- ✗ Not maintaining test data.

Pro Tips

- ☆ Keep tests independent.
- ☆ Use meaningful locators.
- ☆ Leverage trace & UI mode.
- ☆ Run in parallel wisely.
- ☆ Review reports regularly.



Right tools. Smart configs. CI/CD power. Ship quality with confidence!

100